Git and GitHub

Your Downloads Folder

lab3.c
lab3_v2.c
lab3_final.c
lab3_final_FIXED.c
lab3_final_FIXED_real.c
lab3_final_use_this_one.c

Which one compiles?
What changed? Who wrote it?

You Have Already Invented This

Keeping dated copies **is** *version control*

You do it because you need to:

go back to a version that worked
remember what you changed
share the work with somebody else

You just do not have a good tool for it yet

What Version Control Is

A system that records **snapshots** *of your whole project over time*

So that you can:

compare any two snapshots
return to any earlier snapshot
combine work from many people

Three Ways to Picture It

A **save point** *in a game*

reach a checkpoint, reload it if things go badly

A **lab notebook**

dated entries saying what you did and why

Document history

but for an entire project of files at once

What Git Gives You

A **complete history**

every version of every file, with a note why

A **working undo button**

return to any earlier state in one command

Safe experiments *and* **real collaboration**

more on both of these later

Git is not GitHub

These are two different things

People use the names as if they were one

That confusion causes real trouble later

So let us be precise

Git

A **program on your computer**

Free and open source, written in 2005 by Linus Torvalds for the Linux kernel

Runs offline - the full history lives in a hidden **.git** *folder inside your project*

You could use git forever and never open a website

GitHub

A **website that hosts git repositories**

Stores a copy of your repo in the cloud
this copy is called a **remote**

Adds collaboration: pull requests, review, issues

Adds automation: run tests on every push

Alternatives: GitLab, Bitbucket - same git underneath

An Analogy

Git is to GitHub

what a **camera** *is to* **Instagram**

One makes the thing

The other is where you share it

Installing git

macOS (comes with the Xcode command line tools):

git --version

Windows:

download from git-scm.com, then use **Git Bash**

Linux (Debian, Ubuntu):

sudo apt install git

Tell git Who You Are

Do this once, on each machine you use:

git config --global user.name "Ada Lovelace"

git config --global user.email "ada@univ.edu"

git config --global init.defaultBranch main

Check it with: *git config --list*

Why the Name Matters

Every commit you make is **stamped** *with that name and email*

It is how GitHub knows the work is yours

It is what a future employer sees when they look at a year of your contributions

Use the email you register with GitHub

Starting Your Own Repository

mkdir ee101-lab1
cd ee101-lab1
git init

This creates the hidden **.git** *folder*

That folder **is** *the repository*

Use this when the project starts on your machine

Copying an Existing Repository

git clone https://github.com/eecs101/lab1.git
cd lab1

You now have the **full history**, *not just the files*

Use this for starter code posted for the course

Clone once. Do not clone again to get updates - use git pull (later in this lecture)

Three Places Your Work Lives

Almost every git command moves your changes from one of these places to the next



A fourth place once you use GitHub: the remote

The Working Directory

The files you actually **edit**

Your folder on disk

Edit, delete, rename - git notices
but does nothing about it yet

Nothing here is safe until you commit

The Staging Area

The changes you have **chosen**

Think of a shopping cart

You pick exactly which edits belong in the next snapshot

This is why git has two steps instead of one

The Repository

The **permanent history**

The hidden .git folder

Once a snapshot is committed here it is safe and you can return to it

A fourth place appears once you use GitHub: the **remote**

The Everyday Loop

1. *git status* — *what changed?*
2. *git add* — *choose the changes*
3. *git commit* — *save the snapshot*
4. *git push* — *send it to GitHub*

You will run this a thousand times

git status

Run it constantly

It tells you what git currently sees:

files you changed
files you staged
files git is not tracking at all

It usually suggests the command you need next

git add

git add main.c

Moves that change into the staging area

git add . *stages everything in the folder*

Prefer naming the files - it keeps you honest about what you are about to commit

git commit

git commit -m "Fix off-by-one in ADC loop"

Takes everything in the staging area and records it as a permanent snapshot

The **-m** *flag supplies the message*

Leave it off and git opens an editor

git push

git push

Sends your commits to GitHub

Nothing is backed up until you push

Push at least once per work session

A commit on a stolen laptop helps nobody

What a Commit Is

A commit records four things:

a full snapshot of every tracked file
who made it, and when
a message explaining why
a pointer to the commit before it

That last one is what makes it a **history**

The Commit Hash

a1b2c3d4e5f6...

A 40 character **fingerprint** *of the snapshot*

Change one byte and the hash changes

That is why history cannot be quietly rewritten

In practice you type the first 7 characters

Commit Messages: Bad

stuff

asdf

fixed it

final version FINAL

Useless to you in three weeks

Commit Messages: Good

Add binary search to lookup table

Fix segfault when input file is empty

Refactor timer ISR for readability

Update README with build steps

Writing a Good Message

Finish the sentence:

"This commit will ..."

Present tense

Under about 50 characters

One logical change per commit

Reading Your History

git log --oneline	*compact list of commits*
git log --graph	*picture of branches*
git diff	*what have I changed?*
git diff --staged	*what am I about to commit?*
git show a1b2c3d	*what did that commit change?*

git log --oneline

a1b2c3d Add .gitignore for build files
9f4e21c Fix ADC off-by-one error
7ab03de Add unit test for parser
c58d1f0 Merge branch 'sensor-fix'
2e9b74a Handle empty input file
0d3c6b8 Initial commit

Newest at the top. The short hash on the left
is how you name that commit in other commands

Stuck in the Pager?

If git log fills the screen and will not exit,

press **q**

That is the "less" pager, not a frozen terminal

Space pages down, b pages back

Not Everything Belongs in a Repo

Commit the **source**, *not the* **products**

Leave out:

*build output: a.out, *.o, build/*
huge or binary files: datasets, video
secrets: API keys, passwords, .env
OS and editor clutter: .DS_Store, .vscode/

A .gitignore File

```
# compiled output
*.o
a.out
build/

# environment and OS
.env
.DS_Store
```

Put it in the project root, commit it, forget it

Secrets Are Forever

Once a key is pushed, assume it is **public**

Removing it from history is painful

GitHub scans public repos for keys
and so does everybody else

Never commit a password, token, or private key

How to Undo Things

I want the last committed version back:

git restore file.c

I staged something by mistake:

git restore --staged file.c

My last commit message has a typo:

git commit --amend -m "Better message"

Undoing an Old Commit

A commit from last week broke everything:

git revert a1b2c3d

This makes a **new commit** *that undoes the old one*

Honest and safe - the history still shows what happened and when you undid it

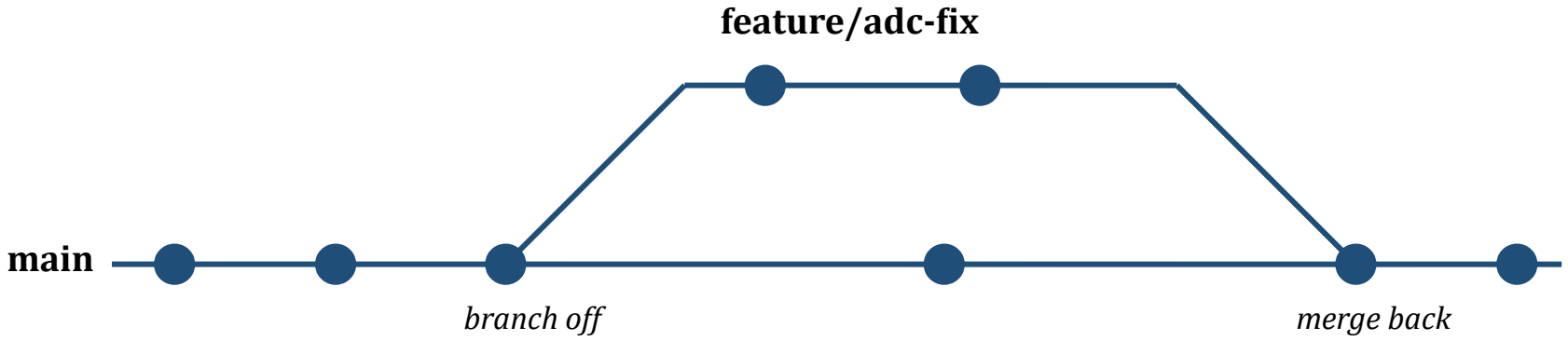
A Branch

A **parallel line of work**

main always holds code that works

Anything new starts on a branch

It merges back when it is ready



Why Branch?

Experiment safely

if the idea fails, delete the branch
main never saw it

Work in parallel

four teammates, four branches, nobody blocked

Review before merging

a branch is what a pull request asks people to read

Branch Commands

Create one and switch to it:

git switch -c feature/adc-fix

See where you are:

git branch

Go back to main:

git switch main

Switching Changes Your Files

The folder on disk **rewrites itself** *to match the branch you switched to*

Commit before you switch

Name branches usefully:

feature/parser, fix/segfault, hw3-part2

not "test" or "new"

Merging

Stand on the branch that **receives** *the work:*

```
git switch main
git merge feature/adc-fix
```

Then tidy up:

```
git branch -d feature/adc-fix
```

Merge Conflicts

Git merges automatically

Until two people change the **same lines**

Then it stops and asks you to decide

This is normal, not a disaster

It means git is protecting you

What a Conflict Looks Like

```
<<<<<<< HEAD
int limit = 100;
=======
int limit = 250;
>>>>>>> feature/adc-fix
```

Your version is on top
Their version is below

Resolving a Conflict

1. *Open the file (git status lists them)*

2. *Choose the correct code - yours, theirs,*
   or something new that combines both

3. *Delete the <<<<<<<, =======, >>>>>>> lines*

4. *git add the file, then git commit*

What GitHub Adds

Off-site backup

laptop stolen the night before a deadline?
the work is safe

Pull requests

propose changes, discuss them, then merge

Issues

a bug tracker attached to the code itself

What GitHub Adds

Code review

teammates comment on exact lines before merging

Actions

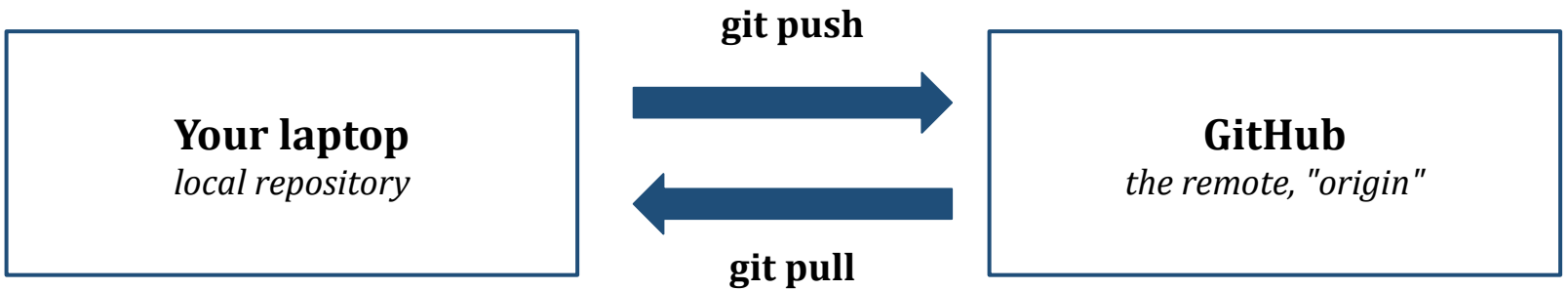
build and test automatically on every push

A public portfolio

employers read GitHub profiles, and yours starts in this course

Your Repo and the Remote

Your laptop holds the whole repository
GitHub holds a copy called the **remote**



Both copies hold the full history

Publishing a Local Repository

Create an empty repository on github.com first, then:

git remote add origin https://github.com:you/lab1.git

git branch -M main

git push -u origin main

Getting Other People's Work

git pull	*fetch and merge new commits*
git fetch	*download them, do not merge yet*
git remote -v	*show which URLs this repo uses*

Pull before you start working each day

Signing In: SSH Keys

GitHub stopped accepting account passwords in 2021

ssh-keygen -t ed25519 -C "you@univ.edu"
press Enter to accept the defaults

cat ~/.ssh/id_ed25519.pub

Paste that line into GitHub:
Settings > SSH and GPG keys > New

Public and Private Keys

ssh-keygen makes **two** *files:*

id_ed25519.pub	*the* **public** *key - share it*
id_ed25519	*the* **private** *key - never share it*

Only the .pub file goes to GitHub

Never commit either one to a repository

Signing In: Access Token

If you use https instead of ssh:

Settings > Developer settings > Personal access tokens

Generate a token, then paste it when git asks for a password

The token is shown **once** *- store it safely*

The Team Workflow

1. **Pull** *start from the latest main*
2. **Branch** *one branch per feature or fix*
3. **Commit** *small, well described snapshots*
4. **Push** *send the branch to GitHub*

The Team Workflow

5. **Pull request** *ask the team to review it*

6. **Review** *comments, changes, approval*

7. **Merge** *the branch joins main*

8. **Repeat** *everyone pulls the new main*

Golden rule: **never commit directly to main**

Pull Requests

A request to merge your branch

Opened on GitHub after you push a branch

Shows every commit and a line by line diff

Teammates comment, request changes, approve

A Merge button brings the work into main

Forks

Your own copy of somebody else's repository

One click on GitHub

Clone your fork, branch, commit, push as normal

Then open a pull request back to the original

How you contribute without write access

Your Profile is a Resume

Recruiters and graduate advisors look at it

Four years of coursework and projects is a compelling record

Write a **README** *for every project:*
what it does, how to build it, a screenshot

Ten minutes that doubles the value of the repo

Free Tools for Students

The **GitHub Student Developer Pack**
education.github.com/pack

Professional tools, cloud credits, and more

GitHub Pages *hosts a web page free, straight from a repository*

Perfect for a personal or project page

Academic Integrity

Coursework repositories must be **private**
while the course is running

A public solution that another student copies
can make **you** *responsible for a violation*

Check the syllabus

When in doubt, ask me first

Mistakes to Avoid

One commit at the very end

commit every time something works

Committing the whole build folder

write a .gitignore in the first five minutes

Editing files on github.com

it works, but your local copy falls out of date

Mistakes to Avoid

Pushing secrets or keys

removing them from history is painful

Panicking at a conflict

read the markers, choose the lines, commit

Running commands you do not understand

especially reset --hard and push --force

Cheat Sheet: Setup

git config --global user.name "..."	*set your name*
git config --global user.email "..."	*set your email*
git init	*start a repo here*
git clone <url>	*copy a repo*
git remote add origin <url>	*link to GitHub*

Photograph this slide

Cheat Sheet: Every Day

git status	*what changed?*
git add <file>	*stage a change*
git commit -m "msg"	*save a snapshot*
git push	*send to GitHub*
git pull	*get other people's work*
git log --oneline	*see the history*
git diff	*see your changes*

Cheat Sheet: Branch and Fix

git switch -c <name>	*new branch*
git switch main	*change branch*
git merge <name>	*merge it in*
git branch -d <name>	*delete a branch*
git restore <file>	*discard your edits*
git revert <hash>	*undo a commit*

When something goes wrong, run **git status** *first*

Where to Learn More

learngitbranching.js.org

a visual game for branching and merging

git-scm.com/book

the free Pro Git book - chapters 1 to 3

training.github.com

one page cheat sheets

And always: git help <command>

It Becomes Automatic

You now know more git than most people use in a week

What is left is **habit***:*

commit small
push often
branch for anything risky

Let's use git

Exercise: Using **gitpractice**

Make a repository, commit, branch, merge, push